

ARQUITECTURA, SERVICIOS, INFRAESTRUCTURA Y EJECUCIÓN DEL SOFTWARE:

OpenS – Asistente IA Medico

=====

1. VISIÓN GENERAL DEL SISTEMA

OpenS es una aplicación de asistente médico que combina:

- **Backend** en Python con **FastAPI** (backend-ia/main.py).
- **Frontend** en **Next.js 15** con React (frontend/).
- **Integraciones externas opcionales:**
 - Google Gemini (vía google-generativeai) para IA generativa.
 - Supabase (PostgreSQL) para historial médico de pacientes.

Está pensada principalmente para:

- Uso **local / desarrollo** (puertos 8001 y 5000).
- Futurable a entorno de producción desplegando backend y frontend en infra propia o cloud.

=====

2. ARQUITECTURA LÓGICA DE ALTO NIVEL

2.1. Componentes principales

- **Cliente (Frontend)**
 - Aplicación React con Next.js App Router.
 - Pantalla única de chat + componentes de UI (búsqueda de paciente, timeline, toasts).
 - Comunica con el backend vía HTTP:
 - Ruta API interna en Next: /api/v1/assistant/ask.
 - Esta ruta actúa como **proxy** hacia el backend FastAPI.
- **Servidor de Aplicaciones Backend (FastAPI)**

Expone endpoints:

 - GET / (health simple).
 - GET /health (health detallado).
 - GET /api/v1/models (lista modelos Gemini, si está instalado y configurado).

- POST /api/v1/assistant/ask (endpoint principal del asistente médico).

Lógica:

- Valida el input (id de paciente, texto de la consulta).
- Opcionalmente consulta Supabase para historial del paciente.
- Construye un prompt controlado y llama a Google Gemini (si está disponible).
- Devuelve una respuesta estructurada al frontend.
- **Servicios gestionados externos (opcionales)**
- **Google Gemini (Google Generative AI):**
 - Llamado vía google-generativeai.
 - Modelo configurable por GEMINI_MODEL (por defecto: gemini-2.5-flash).
 - Se usa para generar la respuesta del asistente, bajo un prompt muy restringido.
- **Supabase (PostgreSQL)**
 - Tabla esperada: medical_records.
 - Se usa para obtener el historial médico del paciente, enriqueciendo el contexto de la IA.

2.2. Flujo de datos

1. Usuario interactúa con la UI del chat en el frontend.
2. Frontend envía un POST a /api/v1/assistant/ask (ruta Next.js API).
3. La ruta de Next reenvía (fetch) la petición a:
 - BACKEND_URL (si está definido en env del frontend), o
 - http://localhost:8001/api/v1/assistant/ask como fallback.
4. El backend:
 - Valida patient_id y prompt_text.
 - (Opcional) consulta Supabase para obtener historial.
 - Construye un prompt en español con reglas estrictas.
 - Si tiene cliente Gemini configurado, llama a la API de Google.
 - Si no tiene Gemini, devuelve un mensaje de fallback.
5. Backend responde JSON { response: string, patient_id: number }.
6. Frontend recibe la respuesta, simula un streaming palabra a palabra y la muestra al usuario.

=====

3. DETALLE DEL BACKEND (FASTAPI)

3.1. Estructura del backend

- Archivo principal: backend-ia/main.py
- Configuración de entorno: .env y .env.example
- Dependencias backend (en pyproject.toml, proyecto raíz):
- fastapi
- google-generativeai
- python-dotenv
- supabase
- uvicorn

3.2. Carga de entorno y configuración básica

- Se usa dotenv.load_dotenv() para cargar variables de .env.
- Variables relevantes:
- GOOGLE_API_KEY: clave de Google AI (secreta, **no debe estar en el repo**).
- GEMINI_MODEL: nombre del modelo (por defecto gemini-2.5-flash).
- SUPABASE_URL y SUPABASE_KEY: URL y clave anónima de Supabase.

3.3. CORS

- Configurado en main.py con CORSMiddleware para permitir:
- http://localhost:3000
- http://localhost:5000
- http://127.0.0.1:5000
- http://127.0.0.1:3000
- Métodos permitidos: GET, POST, OPTIONS.
- Cabezeras: Content-Type, Authorization.

3.4. Manejo de dependencias opcionales: Para hacer el backend más robusto en entornos sin todas las librerías instaladas:

- google-generativeai:
- Se intenta importar.

- Si falla, genai se deja en None.
- Si no existe o no hay GOOGLE_API_KEY, el backend **arranca igualmente**, pero:
- /api/v1/models devuelve error específico si falta la librería o la API key.
- /api/v1/assistant/ask usa una respuesta de fallback que avisa que la IA aún no está configurada.
- supabase:
- Se intenta importar create_client y Client.
- Si falla, create_client se deja en None.
- En fetch_patient_history:
- Si faltan librería o credenciales, se devuelve lista vacía y se loguea una advertencia.
- El backend sigue funcionando, pero sin historial real (solo sin datos).

3.5. Endpoints

- GET /
- Devuelve:
- status: "ok".
- google_ai_configured: True/False según haya cliente Gemini activo.
- supabase_configured: True/False según haya URL+KEY.
- GET /health
- Devuelve:
- status: "healthy".
- google_ai_configured.
- supabase_configured.
- api_version: "v1".
- ai_provider: "Google Gemini".
- model: nombre del modelo si hay cliente, si no null.
- GET /api/v1/models
- Lista modelos de Gemini disponibles (filtrando por los que soportan generateContent).
- Requiere:
- Librería google-generativeai instalada.

- GOOGLE_API_KEY configurado.
- En caso contrario, lanza HTTPException 500 con detalle explícito.
- POST /api/v1/assistant/ask
- Request (Pydantic PatientQueryRequest):
- patient_id: int
- prompt_text: str
- Validaciones:
- prompt_text no vacío.
- patient_id > 0.
- Lógica:
- 1. Llama a fetch_patient_history(patient_id).
- 2. Construye un contexto de texto con el historial (fecha + descripción) si existe.
- 3. Construye system_prompt muy restrictivo en español:
 - Debe responder solo con información que esté en el contexto.
 - Si algo no está en el contexto, debe responder **exactamente**:

"Esa información no está disponible en el historial del paciente."

- Se prohíbe inventar datos o dar consejos médicos.
- 4. Construye full_prompt marcando secciones:
 - --- CONTEXTO DEL HISTORIAL MÉDICO ---
 - --- FIN DEL CONTEXTO ---
 - Pregunta del usuario: ...
- 5. Si google_client existe:
 - Llama a generate_content con temperature=0.7, max_output_tokens=500.
 - Reconstruye el texto a partir de candidates[0].content.parts[*].text.
 - Analiza safety_ratings y prompt_feedback para determinar:
 - Respuesta vacía o bloqueada → devuelve mensajes de error explicativos.
- 6. Si NO existe cliente Gemini:
 - Devuelve mensaje de fallback diciendo que la IA está en configuración y que verifiques la variable GOOGLE_API_KEY.

- Response (Pydantic AssistantResponse):
- response: str
- patient_id: int

3.6. Manejo de errores

- Se diferencian:
- HTTPException de negocio/validación → se relanzan íntegramente.
- Errores genéricos (Exception) → se logea el traceback y se responde con 500 y un mensaje Error processing request:
- Errores específicos de Gemini:
- Se capturan y se devuelven como 500 con Error calling Google Gemini API:
- Errores de seguridad de contenido (Gemini):
- Si el modelo bloquea la respuesta, se devuelven mensajes guiados:
- Por ejemplo: infracción de políticas de seguridad o truncamiento por tokens.

3.7. Arranque del servidor backend

- En main.py:
- Bloque if `__name__ == "__main__":` `uvicorn.run(app, host="0.0.0.0", port=8001)`.
- Puerto: **8001**.
- Bind: 0.0.0.0 (acepta conexiones desde cualquier interfaz de red).

=====

4. DETALLE DEL FRONTEND (NEXT.JS 15 / REACT)

4.1. Estructura principal

- Carpeta: frontend/
- Ficheros:
- `app/page.tsx` → página principal de la app (chat).
- `app/layout.tsx` → layout raíz.
- `app/api/v1/assistant/ask/route.ts` → ruta API interna de Next.js.
- `components/PatientCard.tsx`, `PatientSearchBar.tsx`, `MedicalHistoryTimeline.tsx`, `Toast.tsx`, `TypingIndicator.tsx`.
- `app/globals.css` → estilos globales (Tailwind).

- tailwind.config.ts, postcss.config.js, tsconfig.json, next.config.js.

4.2. Configuración de Next.js (next.config.js)

- outputFileTracingRoot: se ajusta a __dirname para tracing correcto al empaquetar.
- rewrites():
- Ruta: source: '/api/:path*'
- Destino: destination: 'http://127.0.0.1:8000/api/:path*'
- Nota: el backend FastAPI actual corre en puerto 8001, así que este rewrite es más un vestigio o pensado para otra configuración. El flujo real de este proyecto está usando la ruta app/api/v1/assistant/ask/route.ts, que hace un fetch directo a http://localhost:8001.

4.3. Ruta API en Next.js (app/api/v1/assistant/ask/route.ts)

- Método implementado: POST.
- Comportamiento:
 1. Lee el body JSON recibido del cliente.
 2. Determina backendUrl:
 - process.env.BACKEND_URL o
 - 'http://localhost:8001' por defecto.
 3. Hace fetch a \${backendUrl}/api/v1/assistant/ask con:
 - Headers Content-Type: application/json.
 - Body: el mismo body recibido.
 4. Si la respuesta NO es ok:
 - Intenta leer JSON de error.
 - Devuelve un error JSON al cliente Next con status del backend.
 5. Si hay error inesperado (try/catch):
 - Imprime en consola.
 - Devuelve JSON con mensaje de error genérico.
 - Es decir, **Next.js actúa aquí como un proxy fino** y punto de entrada unificado para el frontend.

4.4. Página principal (app/page.tsx)

- Es un componente use client (se ejecuta en el cliente).
- Estado principal:

- messages: histórico de mensajes { role: "user" | "assistant", content: string, isStreaming?: boolean }.
- inputText: texto del input.
- isLoading: si hay petición en curso.
- selectedPatient: paciente actual (mock inicial con id 123).
- toasts: lista de mensajes tipo toast { id, message, type }.
- copiedMessageId: para indicar mensaje copiado.
- showTimeline: bool para mostrar u ocultar MedicalHistoryTimeline.
- Funciones clave:
 - scrollToBottom: asegura scroll al final del chat cada vez que cambian messages.
 - addToast / removeToast: gestión de notificaciones.
 - copyToClipboard: copia contenido de mensaje del asistente y muestra toast de éxito o error.
 - streamText(text, messageIndex):
 - Simula streaming: va añadiendo palabras (text.split(" ")) cada 50 ms.
 - Actualiza el mensaje en messages[messageIndex] y al finalizar marca isStreaming=false.
- handleSendMessage:
 1. Valida que inputText no esté vacío.
 2. Añade mensaje del usuario.
 3. Limpia input y pone isLoading=true.
 4. Envía petición:
 - Primero intenta fetch("/api/v1/assistant/ask", ...) (la ruta Next interna).
 - Si eso falla (catch):
 - Probablemente por no estar configurada la ruta, hace fallback a:
 - fetch("http://localhost:8000/api/v1/assistant/ask", ...).
 - Nota: aquí hay una pequeña inconsistencia con el puerto 8001 del backend; el fallback apunta a 8000.
 5. Si la respuesta no es ok:
 - Intenta leer JSON y mostrar mensaje de error detallado.
 6. Si la respuesta es válida:

- Verifica que data.response exista.
- Inserta un mensaje vacío del asistente.
- Desactiva isLoading.
- Llama a streamText(data.response, assistantMessageIndex).
- Muestra toast de éxito "Respuesta recibida".

7. Si hay error (catch):

- Muestra toast "Error: ...".
- Añade un mensaje de error del asistente:

"Lo siento, encontré un error: ... Por favor, verifica que el servidor esté funcionando e intenta de nuevo."

- handleSelectPatient:
- Cambia selectedPatient.
- Muestra toast "Paciente seleccionado: ...".
- Renderizado:
- Cabecera OpenS.
- Barra de búsqueda de paciente (PatientSearchBar).
- Panel de chat (mensajes, indicador de "TypingIndicator" cuando isLoading).
- Input de texto + botón "Enviar" con spinner.
- Botón para mostrar/ocultar historial médico.
- Panel lateral con PatientCard de pacientes recientes.

4.5. Componentes de soporte

- MedicalHistoryTimeline:
- Usa datos mock (no conectados aún al backend/Supabase).
- Muestra una línea de tiempo con iconos coloreados por tipo:
- Rojo → cardiología.
- Azul → prescripción.
- Morado → laboratorios.
- Permite expandir/colapsar descripción larga con botones "Ver más"/"Ver menos".
- Solo muestra historial si patientId === 123; si no, muestra mensaje "Aún no hay registros médicos disponibles".

- Toast, TypingIndicator, PatientCard, PatientSearchBar:
- Mejoran UX con notificaciones, indicador de escritura, tarjetas de pacientes, y búsqueda.

4.6. Ejecución del frontend

- Scripts en frontend/package.json:
- npm run dev → next dev -p 5000.
- npm run build → next build.
- npm run start → next start -p 5000.
- Puerto por defecto de desarrollo: **5000**.

=====

5. INFRAESTRUCTURA Y DESPLIEGUE

5.1. Entorno de desarrollo local

- Sistema operativo: Windows (PowerShell).
- Backend:
- Se levanta con python main.py en backend-ia/.
- Escucha en 0.0.0.0:8001.
- Frontend:
- Se levanta con npm run dev en frontend/.
- Escucha en http://localhost:5000.
- Comunicación entre servicios:
- Frontend → Next API route /api/v1/assistant/ask.
- Next API → Backend FastAPI (http://localhost:8001/api/v1/assistant/ask).

5.2. Dependencias externas

- Google Gemini:
- Requiere:
- Librería google-generativeai.
- GOOGLE_API_KEY válida configurada en .env del backend.
- Conectividad:
- Necesita salida a Internet desde el entorno donde corre el backend.
- Supabase:

- Requiere:
- SUPABASE_URL y SUPABASE_KEY.
- Librería supabase-py.
- Uso actual:
- fetch_patient_history consulta tabla medical_records para enriquecimiento del contexto.

5.3. Seguridad de secretos

- .env (backend) contiene:
- Clave de Google.
- Credenciales Supabase.
- **Buenas prácticas** (recomendadas):
- Nunca commitear .env al repo.
- Rotar claves si se filtraron.
- Limitar permisos de las claves:
- Supabase anon key con permisos mínimos.
- API key de Google con restricciones por host/proyecto.

=====

6. EJECUCIÓN PASO A PASO DEL SOFTWARE

6.1. Preparación

1. Instalar dependencias backend (idealmente en un entorno virtual):
 - Según pyproject.toml, por ejemplo:
 - `pip install fastapi google-generativeai python-dotenv supabase uvicorn`
2. Instalar dependencias frontend:
 - En frontend/:
 - `npm install`.
3. Configurar entorno backend:
 - Crear backend-ia/.env con algo como:

GOOGLE_API_KEY=TU_API_KEYGEMINI_MODEL=gemini-2.5-

flashSUPABASE_URL=...SUPABASE_KEY=...6.2. Arranque de servicios

- Backend:

- En terminal:
- `cd backend-ia`
- `python main.py`
- Comprobación:
- `http://localhost:8001/`
- `http://localhost:8001/health`
- Frontend:
- En otra terminal:
- `cd frontend`
- `npm run dev`
- Comprobación:
- `http://localhost:5000`

6.3. Flujo de uso

1. Usuario abre `http://localhost:5000`.
2. Ve el chat, puede seleccionar pacientes y ver historial (mock o real dependiendo de Supabase).
3. Escribe una pregunta en el input del chat.
4. Frontend:
 - Envía POST a `/api/v1/assistant/ask` (Next API).
 - Muestra indicador de "Enviando..." y `TypingIndicator`.
5. Next:
 - Reenvía la petición al backend.
6. Backend:
 - Valida, busca historial en Supabase (si está configurado).
 - Llama a Gemini (si está configurado).
 - Devuelve respuesta o mensaje de fallback.
7. Frontend:
 - Recibe texto de respuesta.
 - Lo reproduce con efecto de streaming.

- Permite copiar texto a portapapeles.
- Muestra toasts de éxito/error.

=====

1. RESUMEN

OpenS es una solución de asistente médico basada en IA que combina:

- **Frontend web** (Next.js / React) para interacción con usuarios clínicos.
- **Backend de APIs** (FastAPI en Python) para orquestar lógica de negocio y acceso a datos.
- **Servicios gestionados externos:**
 - Google Gemini (Google AI) para generación de lenguaje natural.
 - Supabase (PostgreSQL gestionado) para almacenamiento de historial clínico.

El diseño actual está optimizado para entorno de desarrollo local, pero su arquitectura se puede extrapolar fácilmente a una topología de producción en la nube, basada en microservicio único (monolito modular) para backend, frontend desacoplado y servicios gestionados.

=====

2. ARQUITECTURA LÓGICA

2.1. Capas lógicas

- **Capa de Presentación (Frontend Web)**
 - Implementada con Next.js 15 y React.
 - Compone la interfaz de chat, búsqueda de pacientes, timeline médico y notificaciones.
 - Gestiona UX avanzada: indicadores de escritura, streaming de texto, toasts, copiado al portapapeles.
- **Capa de API / Negocio (Backend FastAPI)**
 - Expone endpoints HTTP REST:
 - / y /health (salud del servicio).
 - /api/v1/assistant/ask (endpoint principal de consulta a la IA).
 - /api/v1/models (gestión y diagnóstico de modelos Gemini).
 - Encapsula la lógica de:
 - Validación de requests.
 - Construcción de contexto clínico (historial).
 - Construcción de prompts seguros para la IA.

- Manejo de errores funcionales y técnicos.
- **Capa de Integración y Datos**
- Integración con:
 - **Google Gemini** a través de SDK google-generativeai.
 - **Supabase** mediante SDK de Python (supabase-py).
- Supabase aporta:
 - Almacenamiento persistente de historial médico (tabla medical_records).
 - APIs gestionadas con autenticación basada en claves.

2.2. Flujos funcionales principales

- **Flujo de Consulta al Asistente Médico**
 1. Usuario ingresa una pregunta en el frontend.
 2. El frontend envía la solicitud a una ruta API interna de Next.js.
 3. La ruta API de Next reenvía la solicitud al backend FastAPI.
 4. El backend:
 - Valida parámetros (ID de paciente, texto).
 - Recupera historial en Supabase (si está configurado).
 - Construye prompt controlado en español (con reglas estrictas de no invención).
 - Llama a Google Gemini, analiza respuesta y estados de seguridad.
 5. El backend retorna un texto generado al frontend.
 6. El frontend presenta la respuesta con efecto de streaming y opciones de copiado.
- **Flujo de Monitorización / Salud**
 - Consultas periódicas (manuales o por monitorización) a:
 - /health → estado interno (IA configurada, Supabase configurado, modelo activo).
 - / → verificación simple de servicio.

=====

3. ARQUITECTURA FÍSICA Y DESPLIEGUE

3.1. Topología actual (desarrollo local)

- **Nodo único de desarrollo** (equipo del desarrollador):
- Contiene:

- Servicio de backend FastAPI escuchando en puerto 8001.
- Servicio de frontend Next.js escuchando en puerto 5000.
- Cliente de base de datos y conexión remota con Supabase (si se configuran URL/KEY).
- Conexión de salida hacia Google Gemini (si se configura GOOGLE_API_KEY).
- Comunicación:
- Frontend → Backend:
- A través de una ruta interna de Next (/api/v1/assistant/ask) que hace fetch hacia http://localhost:8001/api/v1/assistant/ask.
- Backend → Servicios Externos:
- Google Gemini (HTTPs saliente).
- Supabase (HTTPs saliente).

3.2. Topología recomendada para producción (alto nivel)

- **Capa de Frontend**
- Desplegar la aplicación Next.js:
- O bien como **frontend estático** (build + hosting en CDN, por ejemplo Vercel, S3+CloudFront, etc.).
- O bien como **app de servidor Node** ejecutando next start detrás de un reverse proxy (Nginx / API Gateway).
- **Capa de Backend (FastAPI)**
- Desplegar el backend como un servicio independiente:
- Contenedor Docker con Uvicorn/Gunicorn.
- Orquestado por Kubernetes / ECS / App Service / Cloud Run, según plataforma.
- Exponer puerto HTTP interno (por ejemplo 8000/8001) detrás de:
- Un balanceador de carga / gateway (ALB, Nginx Ingress, API Gateway).
- **Capa de Servicios Gestionados**
- Google Gemini:
- Servicio externo (Google Cloud) accesible desde backend vía Internet.
- Configurar redes y firewalls para permitir sólo tráfico saliente necesario.
- Supabase:
- Instancia gestionada en la nube con base de datos PostgreSQL.

- Claves / tokens gestionados como secretos en el orquestador (Kubernetes Secrets, AWS Secrets Manager, etc.).
- **Capa de Seguridad y Gobernanza**
- Gestión centralizada de secretos:
- Clave GOOGLE_API_KEY.
- SUPABASE_URL y SUPABASE_KEY.
- Políticas de CORS y seguridad adecuadas sólo para dominios de frontend productivo.
- Monitorización centralizada de logs y métricas.

=====

4. SERVICIOS LÓGICOS Y RESPONSABILIDADES

4.1. Servicio Frontend Web

- **Responsabilidades clave**
- Presentar la interfaz del asistente médico:
- Área de chat.
- Información del paciente seleccionado.
- Línea de tiempo de historial médico (actualmente con datos mock).
- Orquestrar la experiencia del usuario:
- Manejo de estados de carga (loading, errores).
- Notificaciones (toasts).
- Simulación de streaming de texto.
- Actuar como **cliente ligero** de las APIs REST del backend.
- **Tecnologías**
- Next.js 15 (App Router).
- React (funcional, hooks).
- Tailwind CSS para el diseño visual.

4.2. Servicio Backend FastAPI

- **Responsabilidades clave**
- Exponer API REST estable y versionada (v1).
- Encapsular reglas de negocio:

- Validación de entradas.
- Estructurado del contexto clínico del paciente.
- Reglas de seguridad semántica del prompt (no inventar, limitar a historial).
- Integrar de manera segura:
- Llamadas a Google Gemini.
- Consultas a Supabase.
- **Patrones de robustez implementados**
- Dependencias externas opcionales:
- Si google-generativeai no está instalada o falta la API key, el backend sigue arrancando y responde con mensajes de fallback.
- Si Supabase o su SDK faltan, el sistema opera sin historial, pero sin fallar de forma crítica.
- Manejo detallado de errores:
- Diferenciación entre errores de validación de negocio y excepciones técnicas.
- Manejo de safety_ratings y prompt_feedback de Gemini para comunicar problemas de contenido.

4.3. Servicio de IA (Google Gemini)

- **Rol funcional**
- Proveer capacidades de entendimiento de lenguaje natural sobre las preguntas de los usuarios.
- Generar respuestas en español estrictamente basadas en el contexto clínico suministrado.
- **Restricciones aplicadas desde el backend**
- El prompt instruye de forma explícita:
- No inventar información.
- No agregar consejos médicos más allá del historial.
- Usar una respuesta fija si la información solicitada no está presente en el contexto.

4.4. Servicio de Datos Clínicos (Supabase)

- **Rol funcional**
- Almacenar y exponer historial médico estructurado del paciente.
- Permitir enriquecimiento del contexto que se pasa a la IA.
- **Modelo de datos relevante**

- Tabla estimada: medical_records con campos como:
- patient_id.
- date.
- description.
- El backend construye una lista de entradas en forma de texto legible.

=====

5. INFRAESTRUCTURA, SEGURIDAD Y OPERACIONES

5.1. Gestión de Configuración y Secretos

- Variables de entorno clave:
- GOOGLE_API_KEY → clave sensible, debe estar solo en backend.
- GEMINI_MODEL → nombre del modelo, no sensible.
- SUPABASE_URL, SUPABASE_KEY → sensibles (especialmente la key).
- Buenas prácticas para producción:
- No commitar .env a control de versiones.
- Almacenar secretos en almacenes dedicados:
- AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, etc.
- Inyectar secretos en tiempo de despliegue, no en build.

5.2. Seguridad de red

- Recomendaciones:
- Backend expuesto sólo a través de:
- API Gateway / Load Balancer, con TLS.
- Restringir CORS:
- Permitir solamente dominios productivos específicos.
- Minimizar superficie:
- Bloquear acceso directo público a Supabase salvo a través de políticas adecuadas.
- Asegurar que solo backend posee claves que permiten acceso privilegiado, el frontend no debe tener claves administrativas.

5.3. Observabilidad

- Métricas y logging recomendados:

- Backend:
- Logs estructurados de:
- Errores en llamadas a Gemini.
- Problemas de seguridad de contenido (bloqueos de prompt).
- Errores de conexión a Supabase.
- Métricas:
- Latencia y tasa de éxito de `/api/v1/assistant/ask`.
- Número de llamadas a Gemini por unidad de tiempo.
- Frontend:
- Traza de errores en UX (por ejemplo, errores JS o de red).
- Infraestructura:
- Health checks automáticos sobre `/health`.

5.4. Escalabilidad y Alta Disponibilidad

- **Escalabilidad horizontal**
- Backend:
- Contenerizar FastAPI y escalar réplicas detrás de un balanceador.
- Frontend:
- Servir contenido estático desde CDN, escalable globalmente.
- **Puntos sensibles a escalar**
- Llamadas a Gemini:
- Límite de cuota por API key y por proyecto.
- Monitorizar consumo.
- Base de datos Supabase:
- Asegurar índices adecuados sobre `patient_id`, `date`.

5.5. Cumplimiento y privacidad

- Dado que maneja datos potencialmente clínicos:
- Evaluar normativas aplicables (por ejemplo, GDPR, HIPAA dependiendo de jurisdicción).
- Anonimizar o seudonimizar datos cuando sea posible.
- Limitar persistencia de logs con datos clínicos.

=====

6. EJECUCIÓN Y CICLO DE VIDA DEL SOFTWARE

6.1. Ciclo de arranque en desarrollo

1. Configuración:
 - Instalar dependencias backend y frontend.
 - Crear ficheros .env con claves y URLs apropiadas.
2. Arranque backend:
 - Ejecutar python main.py en backend-ia/.
3. Arranque frontend:
 - Ejecutar npm run dev en frontend/.
4. Pruebas manuales:
 - Comprobar /health.
 - Abrir <http://localhost:5000> y realizar consultas de prueba.

6.2. Ciclo de despliegue en producción (propuesto)

1. Pipeline de CI:
 - Linter, tests, build de backend y frontend.
 - Construcción de imágenes Docker (si se usa contenedores).
2. Pipeline de CD:
 - Despliegue de imágenes en clúster de orquestación.
 - Actualización de rutas de frontend en CDN.
3. Post-despliegue:
 - Smoke tests automáticos sobre /health y un flujo de chat básico.
 - Monitorización de logs y métricas durante el periodo de estabilización.